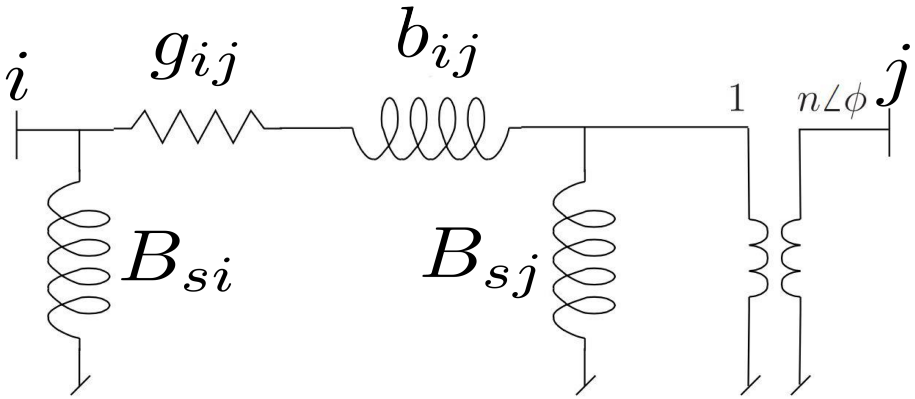
# RAMSES Updates

Petros Aristidou Thierry Van Cutsem

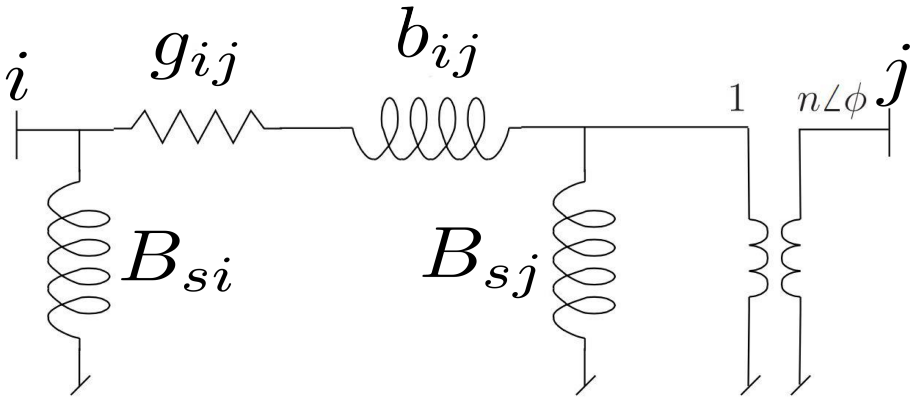
Department of Electrical and Computer Engineering
University of Liège

# Branch series elements or Impetance Loads (IMPLOAD)



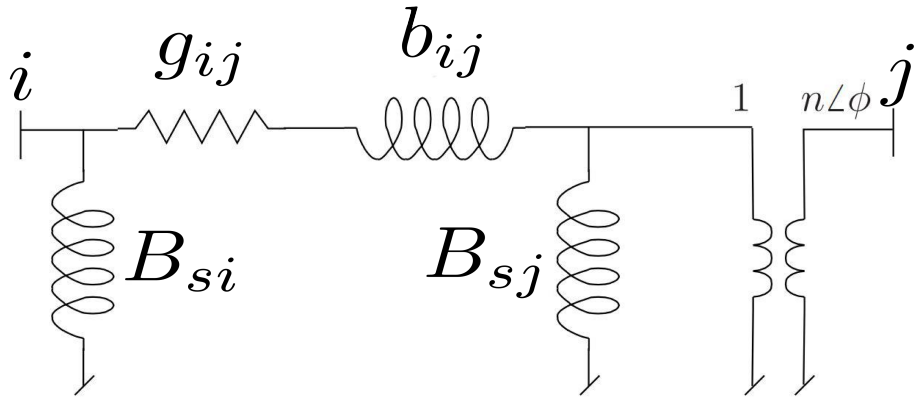

1. $R_b = \dfrac{g_{ij}}{g_{ij}^2 + b_{ij}^2}$ $\qquad X_b = \dfrac{-b_{ij}}{g_{ij}^2 + b_{ij}^2}$

# Branch series elements or Impetance Loads (IMPLOAD)



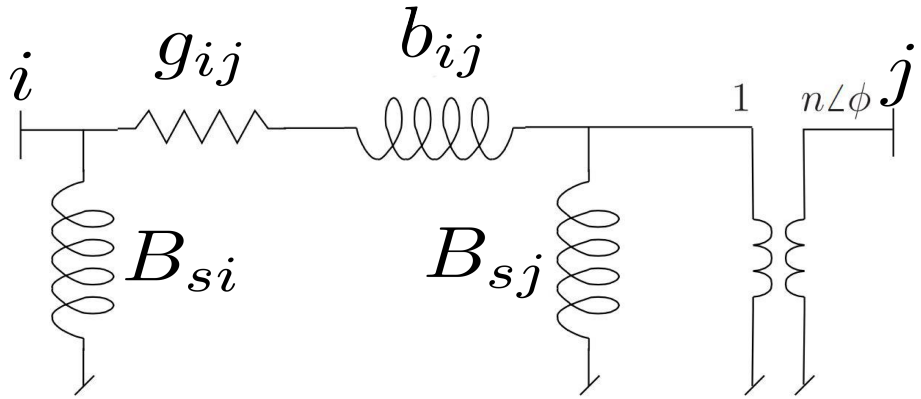

1. $R_b = \dfrac{g_{ij}}{g_{ij}^2 + b_{ij}^2}$ $\qquad X_b = \dfrac{-b_{ij}}{g_{ij}^2 + b_{ij}^2}$
2. if $b_{ij} > 0$ then
   $X_{b-corr} = X_b\,\omega_{COI}$
   else
   $X_{b-corr} = \dfrac{X_b}{\omega_{COI}}$

# Branch series elements or Impetance Loads (IMPLOAD)




1. $R_b = \dfrac{g_{ij}}{g_{ij}^2 + b_{ij}^2}$ $\qquad X_b = \dfrac{-b_{ij}}{g_{ij}^2 + b_{ij}^2}$
2. if $b_{ij} > 0$ then
   $X_{b-corr} = X_b\,\omega_{COI}$
   else
   $X_{b-corr} = \dfrac{X_b}{\omega_{COI}}$
3. $g_{ij-corr} = \dfrac{R_b}{R_b^2 + X_{b-corr}^2}$ $\qquad b_{ij-corr} = \dfrac{-X_{b-corr}}{R_b^2 + X_{b-corr}^2}$

# Branch shunt elements or SHUNTS




if $B_{si} > 0$ then

$$B_{si-corr} = B_{si}\,\omega_{COI}$$

else

$$B_{si-corr} = \frac{B_{si}}{\omega_{COI}}$$

if $B_{sj} > 0$ then

$$B_{sj-corr} = B_{sj}\,\omega_{COI}$$

else

$$B_{sj-corr} = \frac{B_{sj}}{\omega_{COI}}$$

# Solver settings

- Activation: `$NET_FREQ_UPD T/F ;`
- Message: Updating network elements according to COI speed

# Voltage limits

- Definition :

```
DCTL SIM_MINMAXVOLT CTL_NAME VMIN VMAX DEADTIME
STOP_SIMULATION(0/1) ;
```

- Example :

```
DCTL SIM_MINMAXVOLT Warning 0.85 1.10 5 1 ; # when these limits
are crossed, a message will appear and simulation will continue

DCTL SIM_MINMAXVOLT Stopping 0.70 1.20 5 0 ; # when these limits
are crossed, a message will appear and simulation will stop
```

- Execution trace :

```
MESSAGE FROM DCTL sim_minmaxvolt Warning:
t = 124.8000 Voltage out of bounds at bus 1041 : 0.85
STOP CALL FROM DCTL sim_minmaxvolt Stopping:
t = 163.2750 Voltage out of bounds at bus g6 : 0.70
```

# Synchronous machine limits

- Definition :

```
DCTL SIM_SIM_MINMAXSPEED CTL_NAME SPEEDMIN SPEED-
MAX DEADTIME STOP_SIMULATION(0/1) ;
```

# Data structure

```
module ZONES
  use DIMENSIONS

  character (len=20), dimension(mxzon) :: zonname ! names of zones
  integer :: nbzon ! number of zones in simulation
  integer, dimension(mxzon) :: adzonbus ! pointer to list of buses in zone
  integer, dimension(mxzonbus) :: zonbus ! list of buses in zone
  integer, dimension(mxzon) :: adzoninj ! pointer to list of injectors in zone
  integer, dimension(mxzoninj) :: zoninj ! list of injectors in zone
  integer, dimension(mxzon) :: adzonsync ! pointer to list of synchronous machines
in zone
  integer, dimension(mxzonsync) :: zonsync ! list of synchronous machines in zone
end module ZONES
```

# Example of defining and accessing zones

- Defining :

```
ZONE MONTREAL bus1 bus2 bus3 ... ;
```

- Accessing and using :

```
use ZONES
use search_mod, only: searzon
use UDIM, only: inj_model
implicit none
integer i, j, k

call searzon("MONTREAL",k) ! get number of the zone
do i=adzoninj(k),adzoninj(k+1)-1
  j=zoninj(i)
  if(inj_model(j)=="LOAD")then
    injbr(nbsync+j)=0 ! disconnect all injectors of type "LOAD"
  endif
enddo
```